我的景德镇小窑口

一团泥，一炉火，一间自己的小作坊。



瓷器经营游戏设计

在烧窑的间隙，准备下一件作品

第四组 | 张祺睿、李彦波、张若华、熊禹欣、王墨涵

瓷器传播：记住器物，也记住景德镇

让玩家做出的作品，成为继续了解瓷都的起点







识别瓷器

青花、玲珑与红釉的视觉记忆

理解工艺

作品介绍与工序短动画

认识景德镇

作品分享与城市见闻入口

经营为主，节奏为辅

一间作坊，一位学徒，一天比一天顺手

同屏制瓷经营

接单、安排工位、批次烧窑

收工后可选节奏加演



背景故事：回到老街的小窑口

年轻学徒回乡拜师，接过独立看店的第一天



回乡

走进传承多年的陶瓷馆

拜师

从一团泥开始学手艺

开张

用亲手做的瓷器迎接客人

师徒角色设计

经验与好奇心，共同构成游戏的陪伴感



老师傅

沉稳、认真
绿上衣与靛蓝围裙
一句提示，只教一步



陶瓷学徒

年轻、有好奇心
红上衣与米白围裙
以动作呈现学习进步

顾客设计：让瓷器有具体的去处

10位不同顾客，覆盖男女老少与多元族裔



轮廓容易辨认

服饰与姿态区分角色

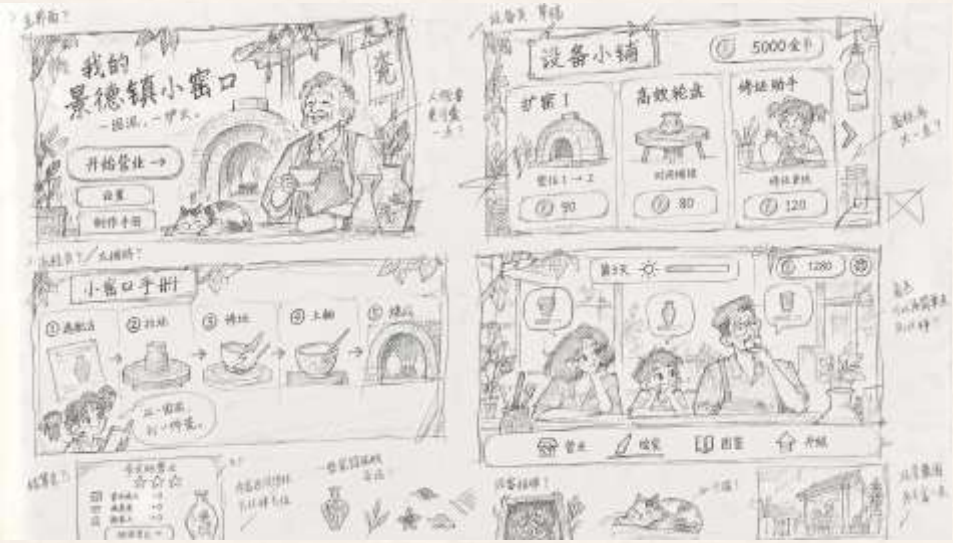
短句交代需求

来买茶具，也带走作品

反馈
接过器物、点头、简短评价

设计手稿迭代

UI与游戏页面概念设计







核心循环：做出作品，再改善作坊

每一次等待，都能成为安排下一件作品的机会

01 看订单，做一件瓷器

02 烧窑时，备下一件

03 交付赚钱，升级设备



收工后，可选“师徒合拍”

经营的乐趣：烧窑时继续备货

从“等它烧好”到“把下一批安排好”



一炉两件

扩窑后一起装入两只杯

烧制10秒

轮盘和修坯台继续工作

客人着急时，少装也能开炉

页面迭代 01：开始页

从功能入口，发展为有角色与作坊氛围的第一印象



早期：结构与功能验证



迭代：师傅、青花与窑火建立识别

页面迭代 02：经营界面

从信息清单，发展为角色与工位共享的作坊场景



早期：工序、库存与订单分区



迭代：场景氛围与角色互动增强

收工加演：和师傅合一段拍子

约24秒的可选表演，为一天留下轻松的结尾



先看示范

轮到学徒时回应

空格 / 点击

动作、声音与拍点同步

可以跳过，营业收益保留

页面体系：设备小铺与收工结算

局外升级解释变化，收工结算呈现成果



设备升级：容量与速度一眼可比



结算反馈：收入连接下一次改善

辅助页面：暂停与设置

随时停下来，也能按自己的节奏继续



暂停：保留当前操作进度



设置：音量、节奏偏移与辅助

升级兑现，连续出货

用一炉多件的变化，呈现“再玩一局”的理由







16—22秒 扩窑升级

容量改变，收益看得见

22—29秒 双杯出窑

匹配顾客，连续交付

29—35秒 师徒与片尾

定格游戏名与开张邀请







让一团泥，成为被带走的作品

我的景德镇小窑口

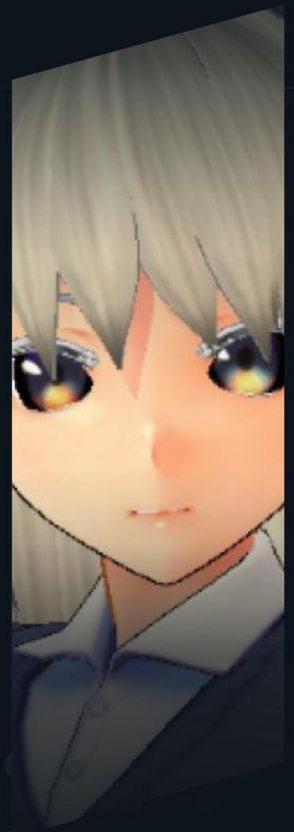
张祺睿、李彦波、张若华、熊禹欣、王墨涵

2026

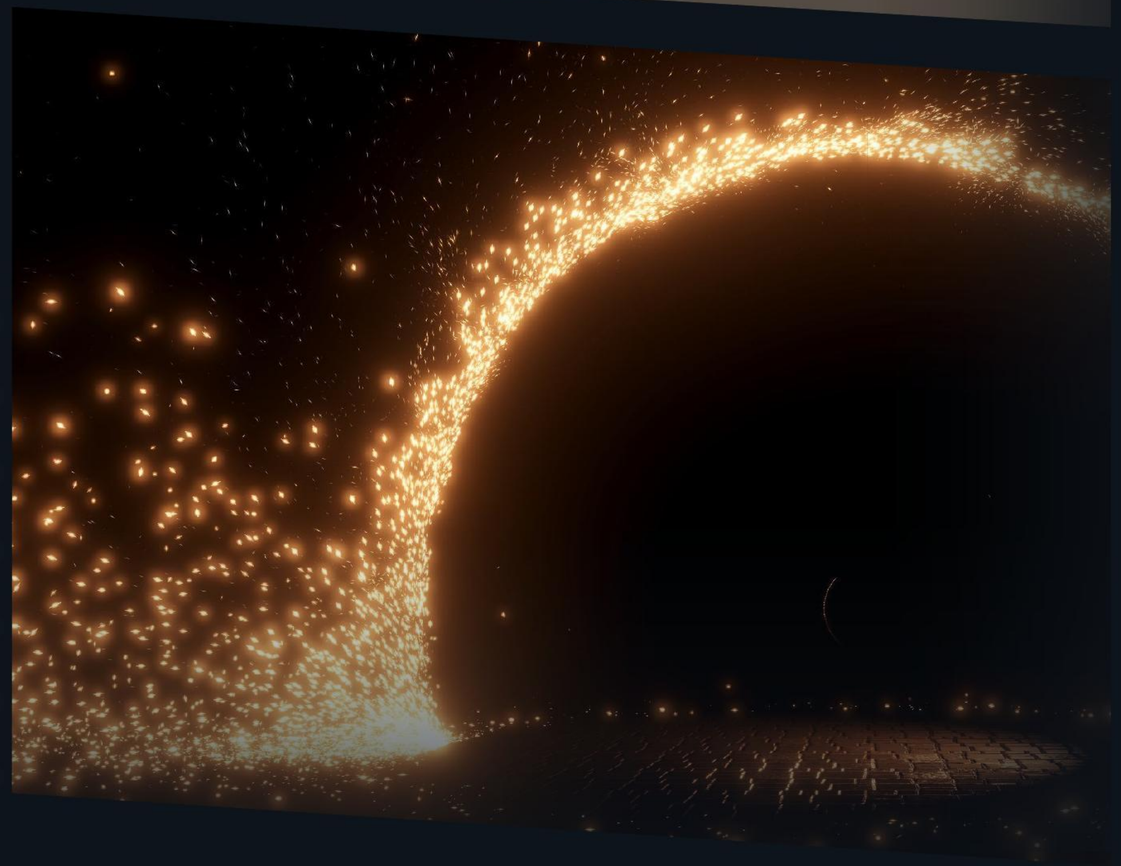
TECHNICAL ARTIST PORTFOLIO

Unity URP Rendering ·
Realtime VFX · Technical Tools

Zhang Qirui
Selected Works
2026







SELECTED WORK

SELECTED PROJECTS

围绕实时渲染、程序化特效与工具开发的三个 Technical Art 案例。

01 STYLIZED CHARACTER NPR RENDERING

Unity URP · HLSL · Face SDF · Hair Shadow

围绕二次元角色渲染，构建 Unity URP NPR Pipeline，解决面部光照、头发遮挡与材质职责分离问题。



02 REALTIME VFX SYSTEMS

VFX Graph · Shader Graph · Runtime Integration

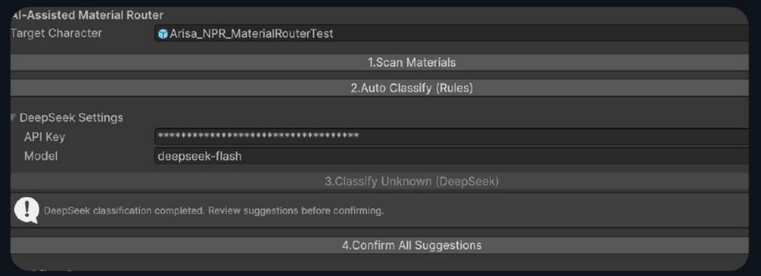
围绕实时技能效果构建 VFX 系统，完成 Portal、Ground Slash 等效果的搭建、结构拆解与 Runtime Integration。



03 AI-ASSISTED MATERIAL ROUTER

Unity C# · Editor Tool · Workflow Automation

面向角色材质整理流程，设计规则优先、人工确认的 Shader Routing 工具。

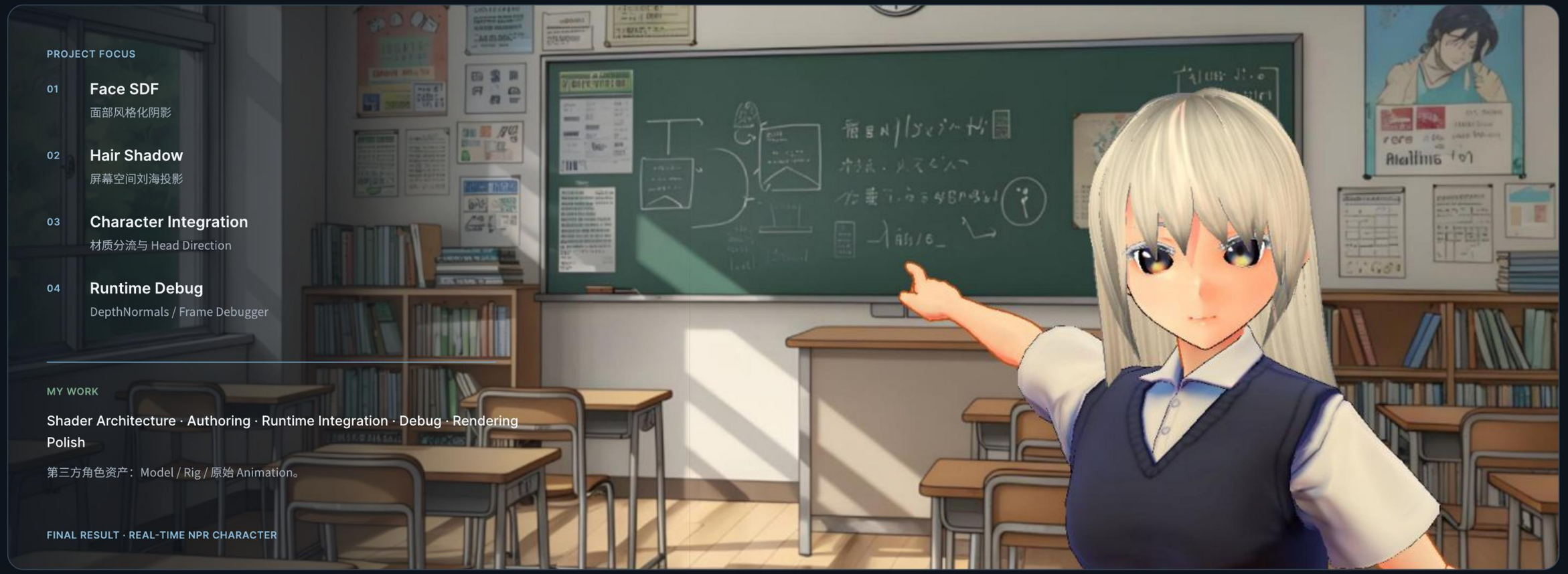


UNITY 6.3 LTS · URP17

NPR Character Rendering

Unity URP 实时 NPR 角色渲染案例

围绕 Face SDF、屏幕空间 Hair Shadow 与材质职责分流，完成从 Authoring、Runtime Integration 到 Debug Verification 的实时角色渲染工作流。



PROJECT FOCUS

01 **Face SDF**
面部风格化阴影

02 **Hair Shadow**
屏幕空间刘海投影

03 **Character Integration**
材质分流与 Head Direction

04 **Runtime Debug**
DepthNormals / Frame Debugger

MY WORK

Shader Architecture · Authoring · Runtime Integration · Debug · Rendering Polish

第三方角色资产：Model / Rig / 原始 Animation。

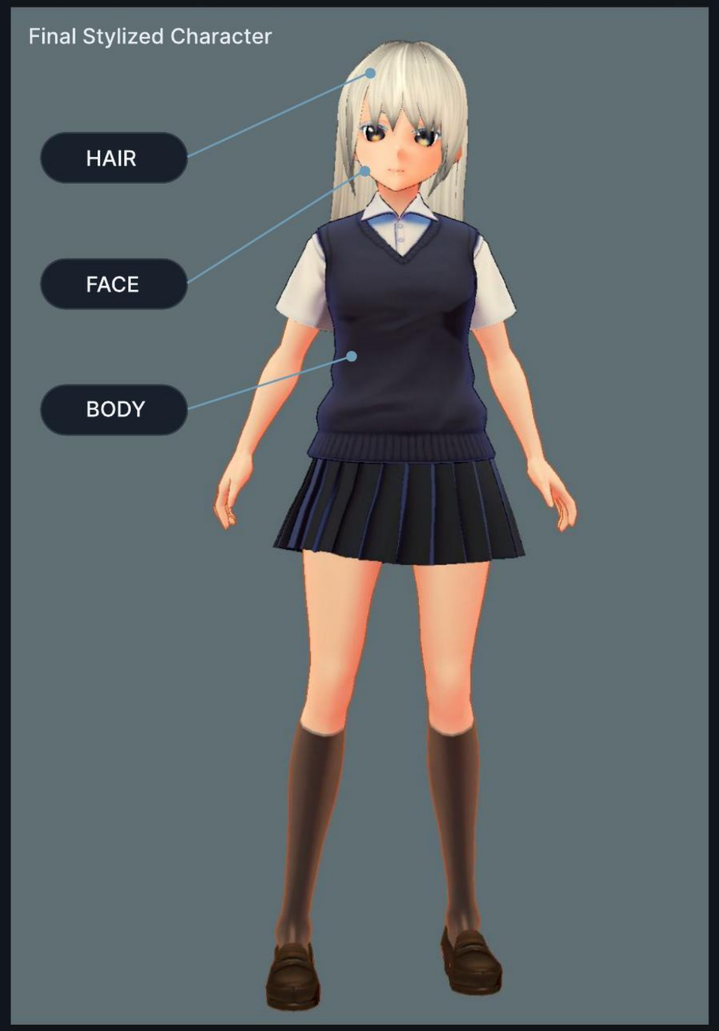
FINAL RESULT · REAL-TIME NPR CHARACTER

TECH STACK

UNITY 6.3 LTS | URP17 | HLSL | C# | BLENDER 4.5

Rendering Architecture

Body / Face / Hair 使用不同的光照职责，针对不同视觉需求拆分 Lighting Model。



HAIR

HAIR LIGHTING

- Toon Diffuse
- Hair Specular
- Rim Lighting
- Alpha Clip

Role

独立处理头发的卡通明暗、方向性高光与发片裁切。

Key Data

- Tangent
- Base Map
- Alpha

FACE

FACE LIGHTING

- Face SDF
- Hair Shadow Detection
- Receiver Mask
- Specular
- Rim Lighting
- Inverted Hull Outline

Role

使用 Face SDF 控制稳定的风格化面部阴影，并叠加屏幕空间刘海遮挡与 Receiver Mask，将几何检测和美术接收区域分离。

Key Data

- Face SDF Map
- Head Direction
- Screen Depth
- Receiver Mask

BODY

TOON LIGHTING

- Toon Diffuse
- Specular
- Rim Lighting
- Inverted Hull Outline

Role

使用通用 Toon Lighting 处理皮肤与服装，保持清晰的明暗分层、高光、Rim 与外轮廓表现。

Key Data

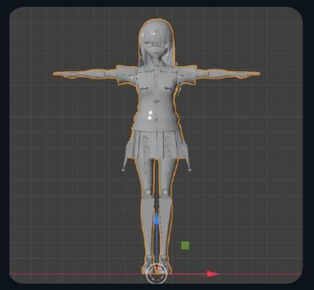
- Base Map
- Occlusion
- Main Light

Face SDF Authoring Pipeline

UNITY URP · HLSL

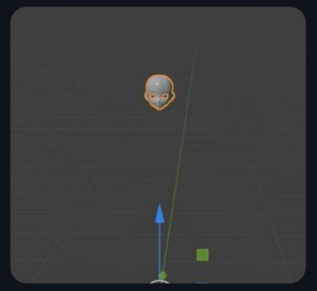
从 Face-only Mesh 到多角度 Shadow Masks，再到运行时阈值比较。

01 Prepare Face Data



Source Mesh

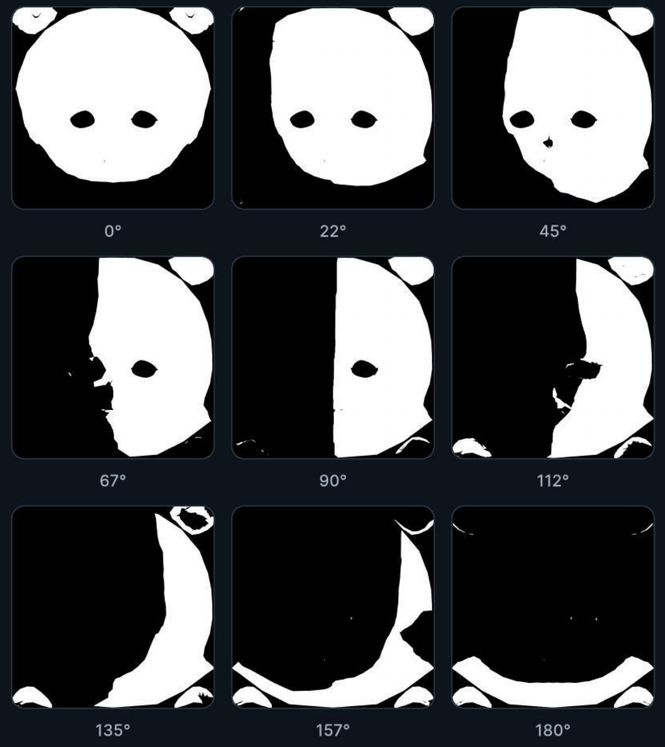
MATERIAL ISOLATION



Face-only Bake Mesh

从单 Mesh / 多 Material 的角色资产中隔离 Face skin 区域，清理其余几何与旧法线数据，为离线 Shadow Bake 准备输入。

02 Bake 9 Angles



0° → 180°：每张图记录同一 UV 下、不同水平光照角度的二值 Shadow Mask。

03 Build Threshold Map



Face Shadow Threshold Map

将多角度 Shadow Masks 转换成逐像素 threshold field。运行时不直接使用 baked lighting，而是读取“当前 texel 在什么 light progress 下进入阴影”。

OPEN-SOURCE TOOL USED

ShadowThresholdMap 用于生成数据；输入准备、Bake、Unity HLSL 集成与 Debug 为本人完成。

04 Runtime Evaluation



Final Runtime Result

RUNTIME LOGIC

Light Direction + Head Basis

→ Face Light Progress

Progress vs. Threshold Map

→ Face Shadow Mask

OFFSET SOFTNESS

Artist Controls：Offset 调整阴影触发位置；Softness 调整 threshold 附近的过渡宽度。

KEY TAKEAWAY

面部阴影不是预烘焙颜色：离线阶段只生成可解释的 threshold data，最终阴影仍由当前 Head Direction 与 Main Light 在 Runtime 中实时决定。

Hair Shadow System

UNITY URP · HLSL

用屏幕空间深度检测刘海遮挡，再用 Receiver Mask 把“几何检测”与“美术接收区域”分离。

01
Screen UV
当前 Face pixel 的屏幕坐标。
它是采样地址，不是 Depth。

02
Light-dir Offset
World Light → View Space，
取 XY 后换算为屏幕像素偏移。

03
Scene Depth
在 offsetScreenUV 采样
Scene Depth，并转成 Eye Depth。

04
Depth Compare
Face Eye Depth − Sampled Eye Depth
得到 depthGap，再用 Bias / Softness 判定。

05
Receiver Mask
raw hair shadow mask × receiver。
只允许指定 Face texel 接收阴影。

06
Independent Composite
保留独立 Hair Shadow Mask，
用 Strength / Color 混合到 baseFaceColor。

RUNTIME EVIDENCE



Receiver ON
ART-DIRECTED RESULT
启用 Receiver Mask，仅允许指定面部区域接收 Hair Shadow。



Receiver OFF
UNRESTRICTED RESULT
关闭 Receiver 限制后，Hair Shadow 会直接作用于更广的面部区域。

红色仅用于可视化未受 Receiver 限制的 Hair Shadow 区域，展示Debug过程，并非最终效果。

DETECTION ≠ ART DIRECTION

GEOMETRY DETECTION

Depth answers “is hair in front?”

Screen UV 只负责“去哪里采样”。
Scene Depth 是被采样出来的标量。

通过 faceEyeDepth 与 sampledEyeDepth 的差值，
判断偏移位置是否存在更靠近 Camera 的遮挡。

OFFSET　DEPTH BIAS　SOFTNESS

ARTIST RECEIVER



Receiver Mask 不负责检测遮挡；
它只定义 Face 上哪些区域允许接收 Hair Shadow。

IMPLEMENTATION

offsetScreenUV
= screenUV + lightDirSS × pixelOffset

depthGap
= faceEyeDepth − sampledEyeDepth

hairShadowMask
= smoothstep(...) × receiver

STRENGTH　COLOR

KEY TAKEAWAY　Hair Shadow 的关键不是“再加一层阴影”，而是把遮挡检测、接收区域和最终合成拆成独立职责，从而同时获得几何可信度与 Artist Control。

UNITY URP · HLSL / C#

Character Integration & Rendering Polish

把独立 Shader 功能接入真实角色资产：用实际 Evidence 证明 Material Routing、骨骼方向驱动与视觉取舍。

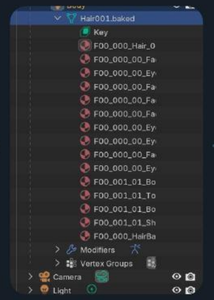
01 Material Routing

单一角色网格



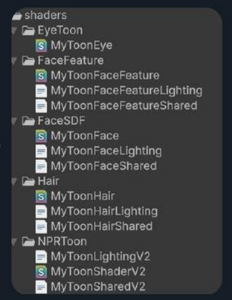
单一角色网格
Single Skinned Mesh

材质槽分布



材质槽分布
Material Slots

Shader职责分流



Shader职责分流
Shader Responsibility Split

Face | Hair | Body | Face Feature | Eye

根据角色区域的视觉需求，将 Material Slots 分流到 Face / Hair / Body / Face Feature / Eye 对应的 Shader 模块。

02 Head Direction Driver

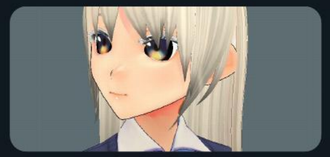
OBJECT MATRIX ≠ SKINNING BONE

Head Bone → C# Driver → MPB → Face Shader

_HeadRightWS · _HeadForwardWS



Head bone neutral



Head bone rotated

只旋转 Head Bone，Root / Camera / Light 保持不变。
C# Driver 读取 Head Bone 朝向，通过 MPB 写入 _HeadForwardWS 与 _HeadRightWS。
因此 Face SDF 的明暗基准随 Head Basis 更新，而不是跟随角色 Root 朝向。

03 Rendering Polish · Evidence & Decision

RUNTIME EVIDENCE

Hair Specular



Specular OFF
Strength = 0



Final
Final setting

Tangent → T·H → Specular Term → Mask → Final Contribution

Hair Specular 由 Tangent 驱动，通过 Mask 控制可见区域，并以较弱权重参与最终结果。

V1 DECISION

Outline Trade-off



Hair Outline OFF
V1 Final Decision



Hair Outline ON
Early / Unified Outline Setting

Hair Outline 会增强头发轮廓，但也更容易放大发片感并抢占面部注意力，因此 V1 最终选择 Reduced / Off

FINAL RESULT

Final Visual Hierarchy



BASE DIFFUSE	PRIMARY
SPECULAR	SECONDARY
RIM / OUTLINE	ACCENT

多个 Lighting Feature 按视觉权重分层，而不是平均竞争注意力。

最终以角色可读性为优先：Base Diffuse 主导形体，Specular 提供材质反馈，Rim / Outline 仅作为轮廓强调。

KEY TAKEAWAY

Integration 的完成点不是“功能存在”，而是数据路由正确、骨骼驱动可信、视觉证据清楚，并能解释为何做出 V1 Trade-off。

Debugging the Data & Render Pipeline

UNITY 6.3 LTS · URP17

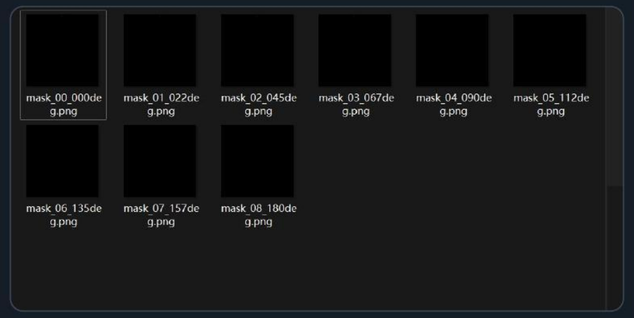
从错误结果定位 Data Producer / Consumer 与 Render Pass，而不是靠随机调参数。

01 Face SDF Bake Failure

AUTHORING · DATA GENERATION

SYMPTOM

9-angle masks became nearly black



Failed bake output

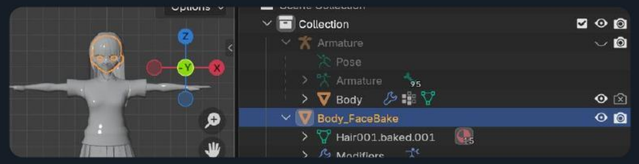
ROOT CAUSE

Render visibility contaminated the bake

Original Body 未关闭 Render Visible，和 Body_FaceBake 重叠参与遮挡。

ORIGINAL BODY · OFF

FACE-ONLY BAKE · ON



Post-fix setup / isolated Face-only input

VALIDATION

0° → 90° → 180° recovers expected progression



0° · mostly white



90° · transition



180° · mostly black

LESSON

DCC visibility state is part of the data-generation pipeline.

错误不在 Threshold 算法本身，而在输入数据生产环境。

02 Hair Shadow Pass Scheduling

RUNTIME · URP PIPELINE

SYMPTOM

Custom Hair produced no usable depth result



DepthOnly path / debug result

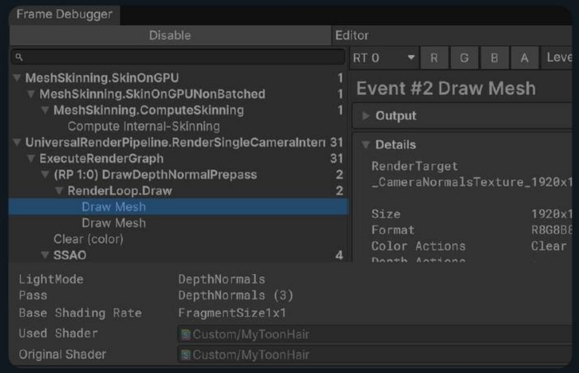


Matching DepthNormals producer

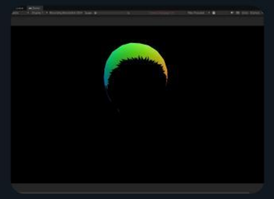
URP/Lit Hair works → Face consumer likely valid → Custom Hair producer / pass path becomes the suspect.

PIPELINE EVIDENCE

Frame Debugger / actual pass execution



FIX · Add DepthNormals Pass



Forward++ + SSAO 需要 Depth + Normal 数据。

Frame Debugger 证明 Hair 已进入 DrawDepthNormalPrepass。

并实际执行 Custom/MyToonHair 的 DepthNormals Pass。

LESSON

Pass exists ≠ Pass executes.

Shader 里“有这个 Pass”不等于运行时“真的执行到这条路径”

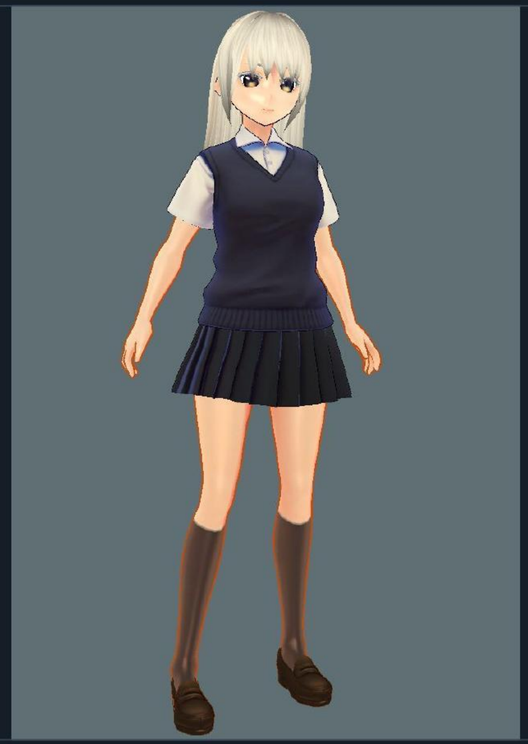
KEY TAKEAWAY

两个 Case 分别落在 DCC Authoring 与 Runtime Pipeline，但共同原则一致：先定位数据在哪里产生、谁消费，再用单变量 A/B 缩小 Root Cause。

UNITY 6.3 LTS · URP17

Scope, Attribution & Next Steps

明确本人负责范围、第三方资产与参考来源，并记录下一阶段可扩展方向。



FINAL CHARACTER · THIRD-PARTY ASSET + CUSTOM NPR RENDERING

角色资产用于承载实时渲染系统；Shader 架构、数据工作流、运行时集成、Debug 与视觉收敛为本人项目职责。

01 My Responsibilities

MY WORK

- **NPR Shader Architecture & Material Routing**
  Body / Face / Hair / Face Feature / Eye 按视觉职责分流。
- **Face SDF Authoring & Runtime Integration**
  完成 Face-only 数据准备、多角度 Bake、Threshold Map 集成与 Runtime HLSL。
- **Hair Specular / Hair Shadow / Receiver Composite**
  完成头发方向性高光、屏幕空间发影检测与接收区域控制。
- **Head Direction Driver & MaterialPropertyBlock Integration**
  用 Head Bone 驱动 Face Shader 的实时 Head Basis。
- **DCC + URP Runtime Debugging / Rendering Polish**
  处理 Bake Visibility、DepthNormals 调度与最终视觉取舍。
- **Inverted Hull Outline、Material Polish 与 Hair Outline V1 Trade-off**
  实现 Inverted Hull Outline，并根据 Hair 的 normals、拓扑与边缘稳定性进行 V1 Reduce / Off 取舍。

02 Assets, References & Open-source Tools

THIRD-PARTY

Arisa Character Package ↗
第三方角色资产；Model / Rig / 原始 Animation / Controller 均非本人制作。

2D Anime - School 00 (FREE) · Hamsters Gaming ↗
P1 Hero Shot 使用的第三方学院背景素材；角色渲染、Shader 与技术集成为本人工作。

REFERENCE

NiloCat · UnityURPToonLitShaderExample ↗
用于学习和参考 URP Toon Lighting 架构，最终 Shader 重构与角色集成为本人完成。

OPEN SOURCE

akasaki1211 · sdf_shadow_threshold_map ↗
用于生成 Face SDF Threshold Map 的开源工具；数据准备、Bake、Runtime HLSL 与 Debug 为本人工作。

ReefSnax · blender-sdf-face-shadow-baker ↗
用于从 Face-only Mesh 烘焙多角度二值 Shadow Masks 的 Blender 开源工具。

03 Tools & Next Steps

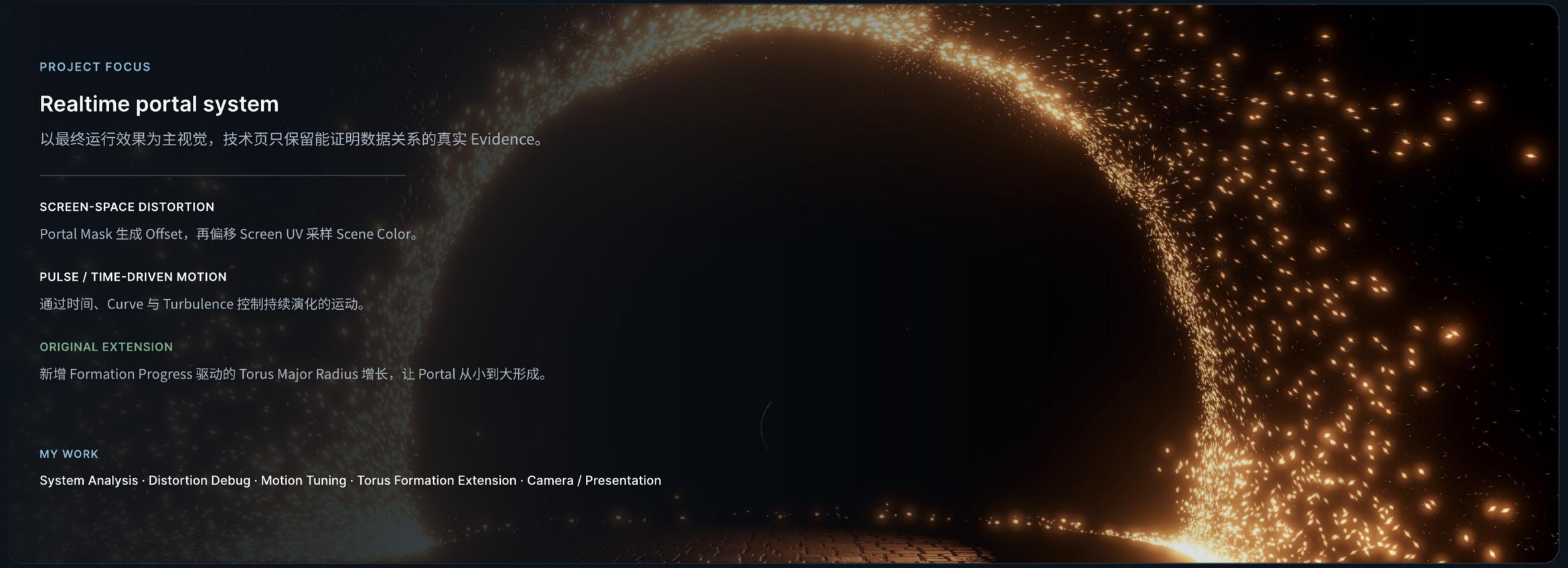
UNITY · HLSL · C# · BLENDER 4.5 · FRAME DEBUGGER · VS CODE

V2 PENDING — Face SDF crossfade · Receiver remap · Authored hair direction

SCOPE — Third-party character asset + reference/open-source tools are explicitly attributed; rendering architecture, authoring workflow, integration, debugging and polish are my work.

Deep Portal VFX

基于 Unity VFX Graph / Shader Graph 的实时传送门拆解、调试与扩展。



PROJECT FOCUS

Realtime portal system

以最终运行效果为主视觉，技术页只保留能证明数据关系的真实 Evidence。

SCREEN-SPACE DISTORTION

Portal Mask 生成 Offset，再偏移 Screen UV 采样 Scene Color。

PULSE / TIME-DRIVEN MOTION

通过时间、Curve 与 Turbulence 控制持续演化的运动。

ORIGINAL EXTENSION

新增 Formation Progress 驱动的 Torus Major Radius 增长，让 Portal 从小到大形成。

MY WORK

System Analysis · Distortion Debug · Motion Tuning · Torus Formation Extension · Camera / Presentation

UNITY · VFX GRAPH · URP · SHADER GRAPH · C#

REFERENCE FOUNDATION

Unity VisualEffectGraph-Samples / Portal ↗

VFX System Architecture

Portal 的核心不是节点数量，而是两条职责清楚的数据路径。

RESPONSIBILITY SPLIT

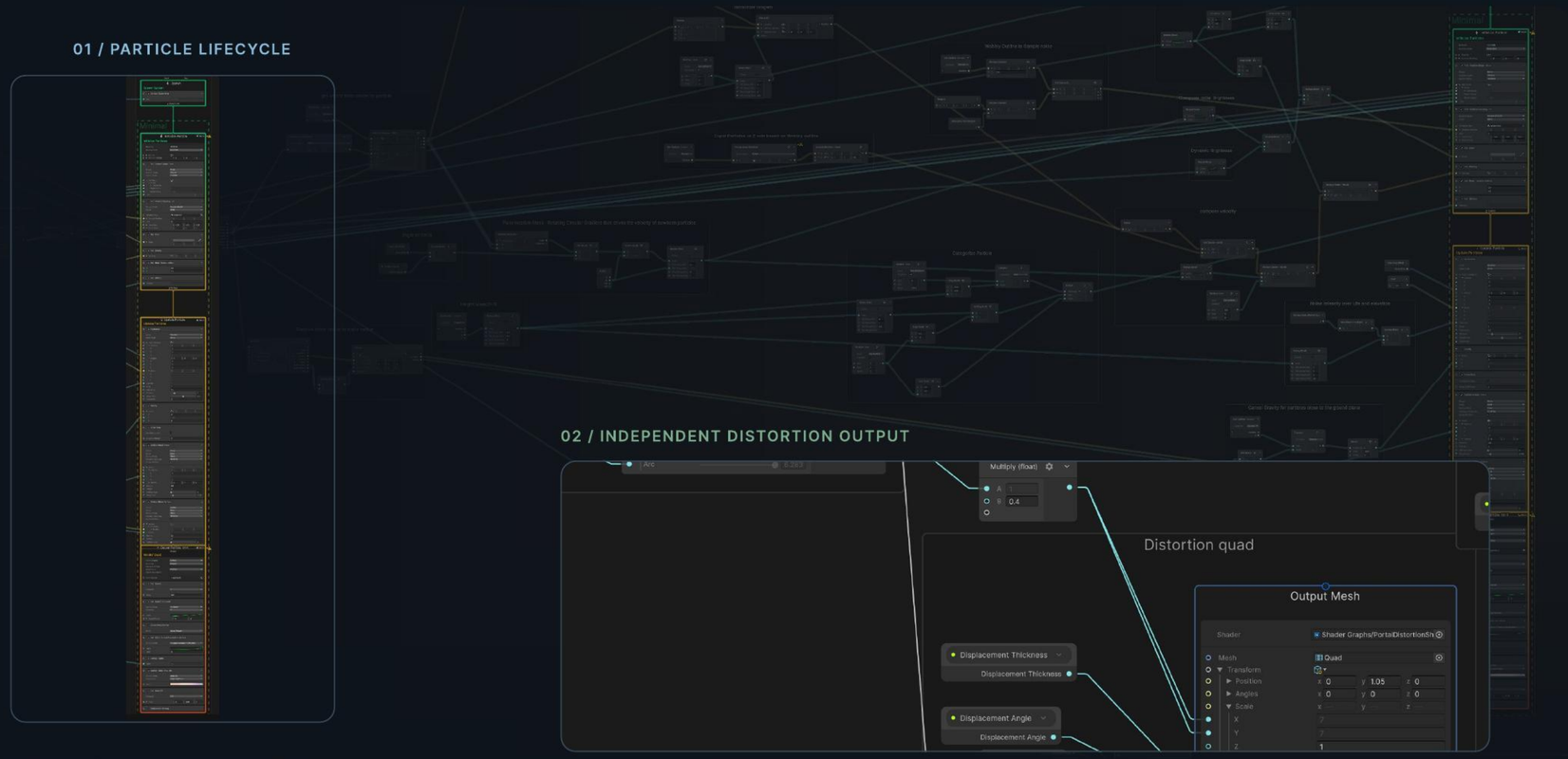
Motion and screen sampling stay separate.

主粒子系统负责生命周期与运动；Distortion Output 独立负责屏幕空间采样。

01 **PARTICLE SYSTEM**
生命周期 / Motion / Render

02 **DISTORTION OUTPUT**
Screen-space sampling / Distortion

GRAPH OVERVIEW



01 / PARTICLE LIFECYCLE

Spawn → Initialize → Update → Output

02 / INDEPENDENT DISTORTION OUTPUT

Mesh Output + Shader Graph · screen-space responsibility

KEY TAKEAWAY 运动逻辑与屏幕采样分责，意味着两条链可以独立调参、验证与 Debug。

Screen-space Portal Distortion

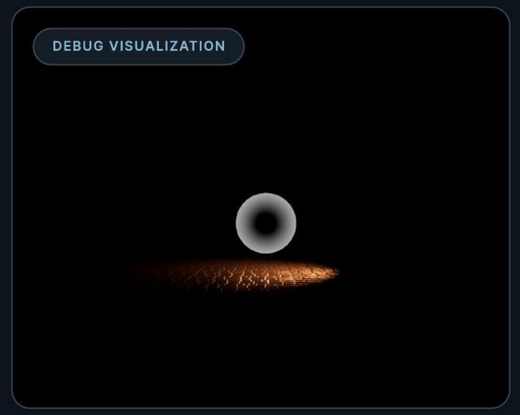
Portal UV 负责塑造扭曲；真正读取 Scene Color 的坐标仍来自 Screen UV。

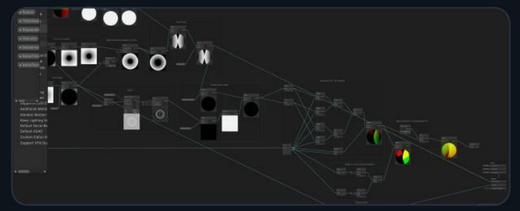
PORTAL MASK → SIGNED 2D OFFSET → SCREEN UV + OFFSET → SCENE COLOR

01

Where distortion is allowed.

灰度强度只描述“哪里 / 多强”，它还不是 Screen UV。



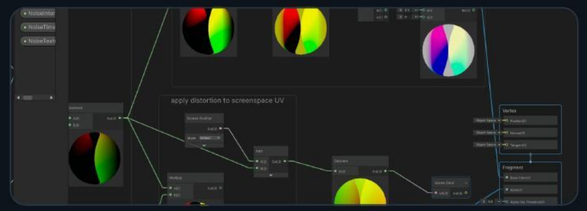


02

The mask becomes displacement.

Offset 是带方向的二维位移；中性值代表零位移。

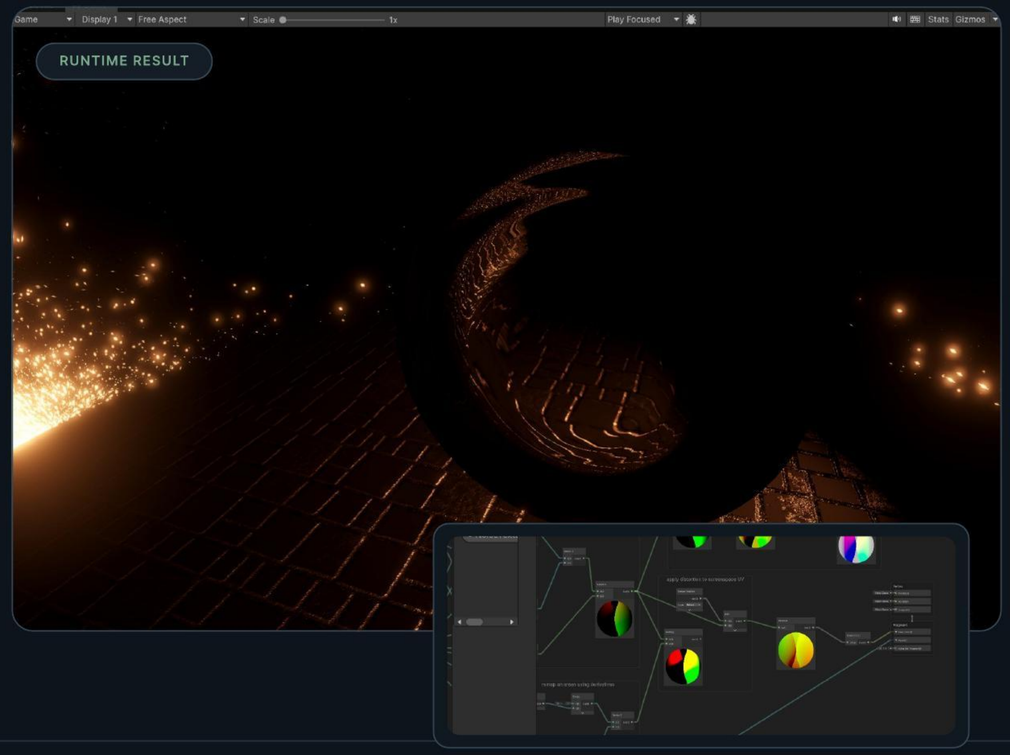




03

Screen-space sampling produces the distortion.

把 Offset 加到 Screen Position 后重新采样 Scene Color，背景才真正发生扭曲。



Screen Position + Offset → Scene Color

KEY TAKEAWAY

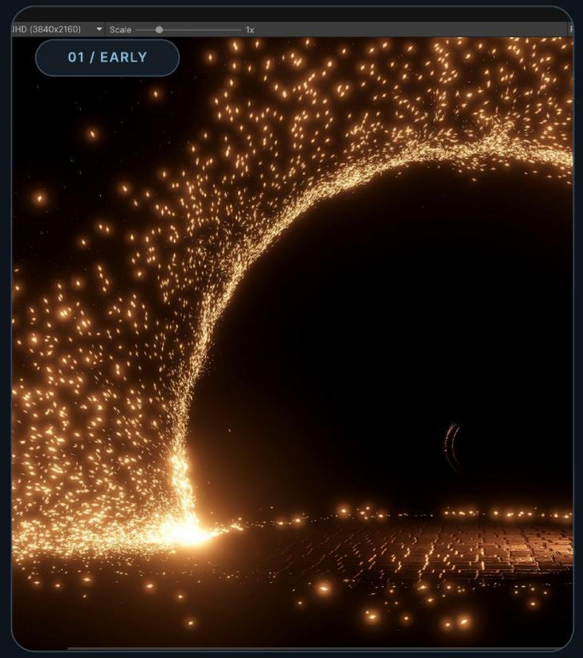
Mask 决定位移，Screen UV 决定采样位置；两种坐标语义不能混用。

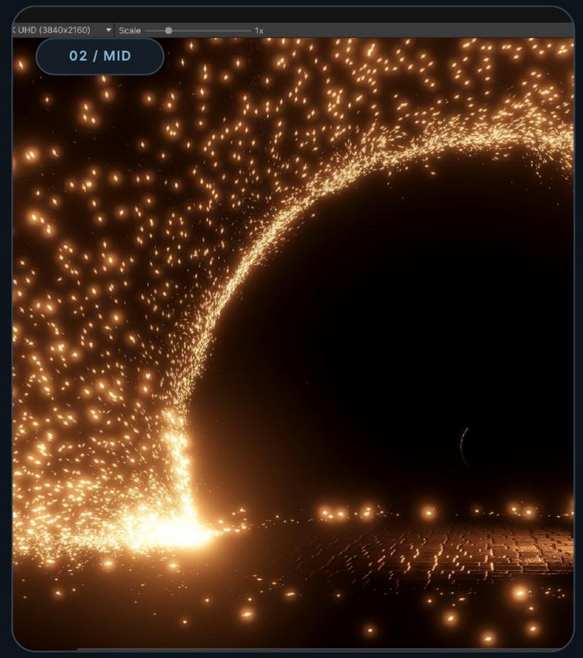
Pulse & Time-driven Motion

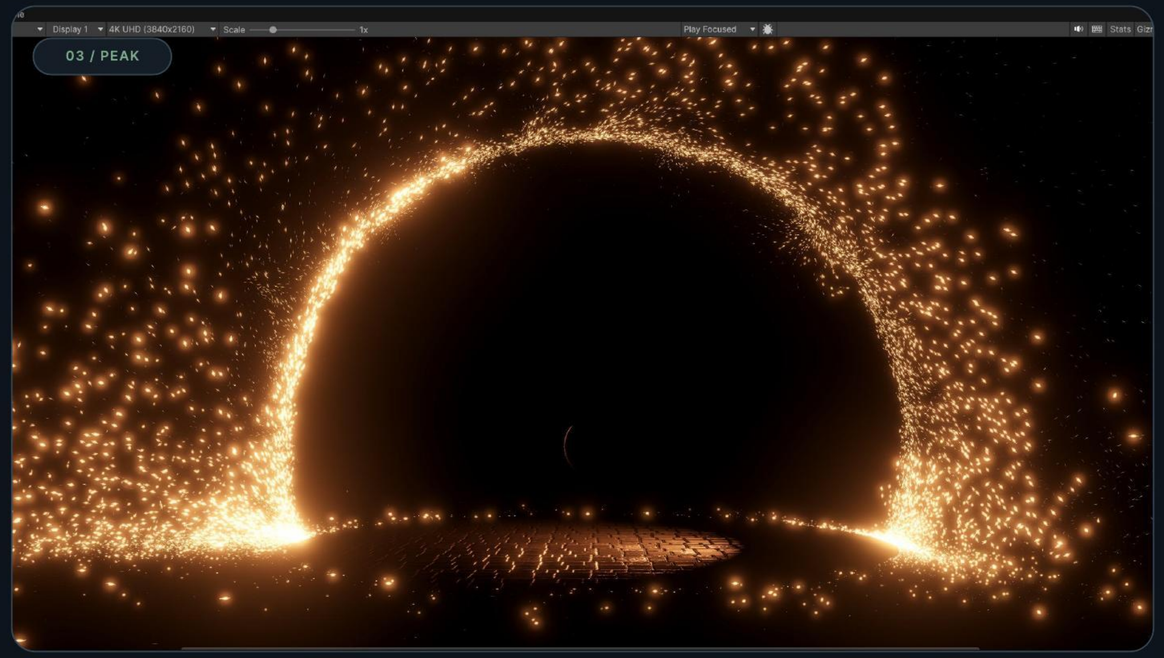
Pulse 组织节奏，Turbulence 让外围运动持续演化。

RUNTIME MOTION SEQUENCE

Same system, three moments.



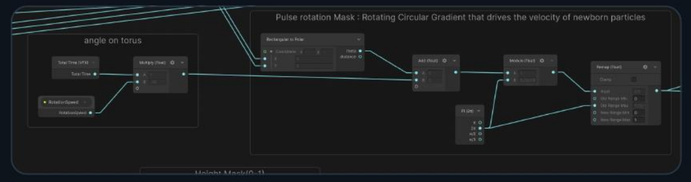


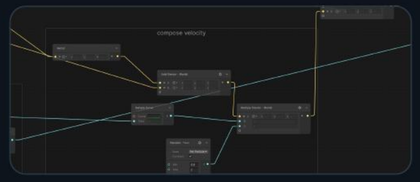


MOTION DRIVER / GRAPH EVIDENCE

Graph 只证明驱动关系；视觉成立由上方 Runtime Result 负责。

PULSE · Phase → Velocity





TURBULENCE · Time → Field Transform



KEY TAKEAWAY

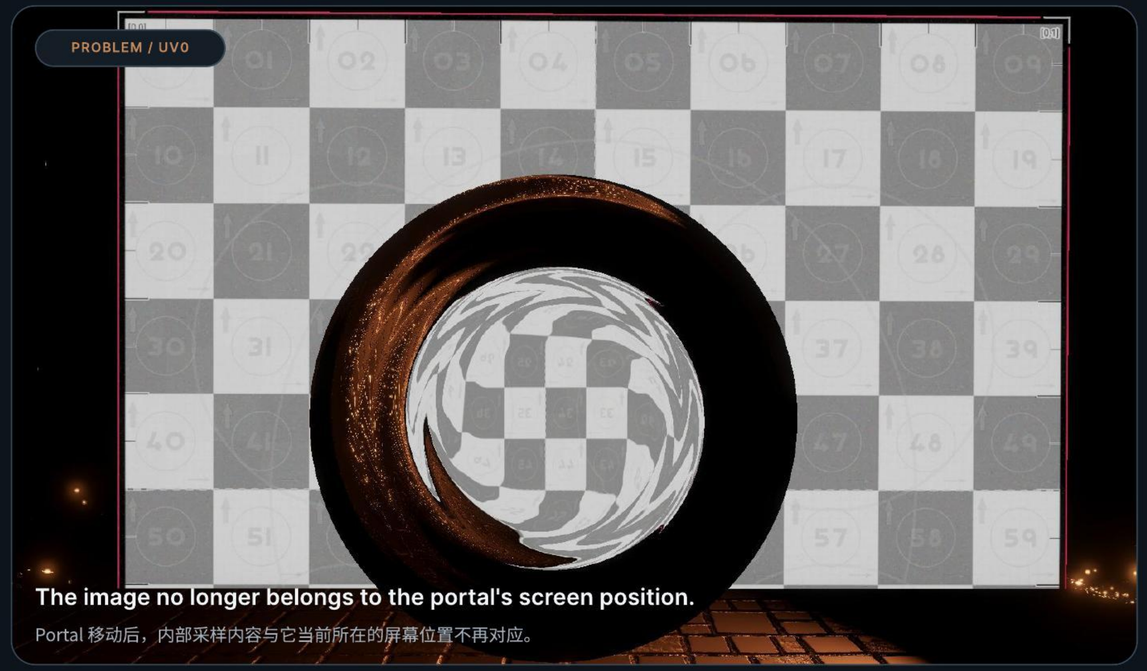
Motion 不是单一 Noise：Pulse 负责节奏，移动中的 Turbulence Field 负责持续演化。

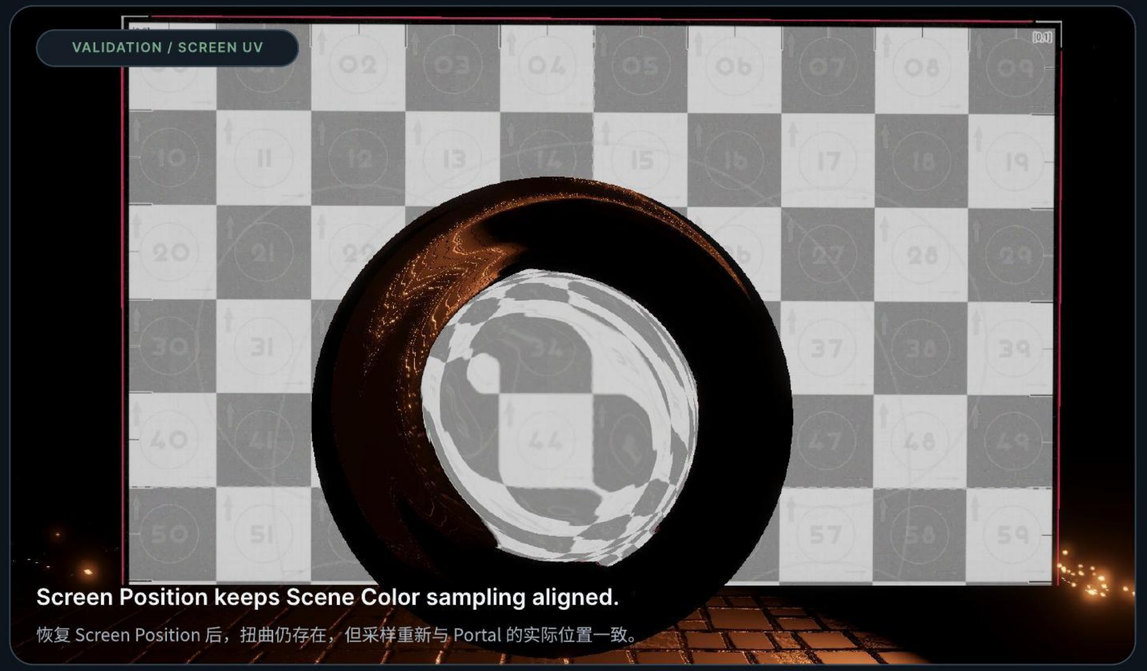
Debugging UV Semantics

扭曲算法本身没有失效；问题来自 Scene Color 使用了错误的采样坐标空间。

RUNTIME EVIDENCE

Same portal. Different coordinate semantics.





ROOT CAUSE / INTERMEDIATE EVIDENCE



WRONG

Portal UV0 + Offset → Scene Color

CORRECT

Screen Position + Offset → Scene Color

LESSON

Numerically valid ≠ semantically correct.

先确认 Producer / Consumer 是否处在同一坐标空间，再怀疑后面的算法。

KEY TAKEAWAY　Portal UV 描述 Mesh 表面；Screen UV 描述屏幕位置。Scene Color 属于后者。

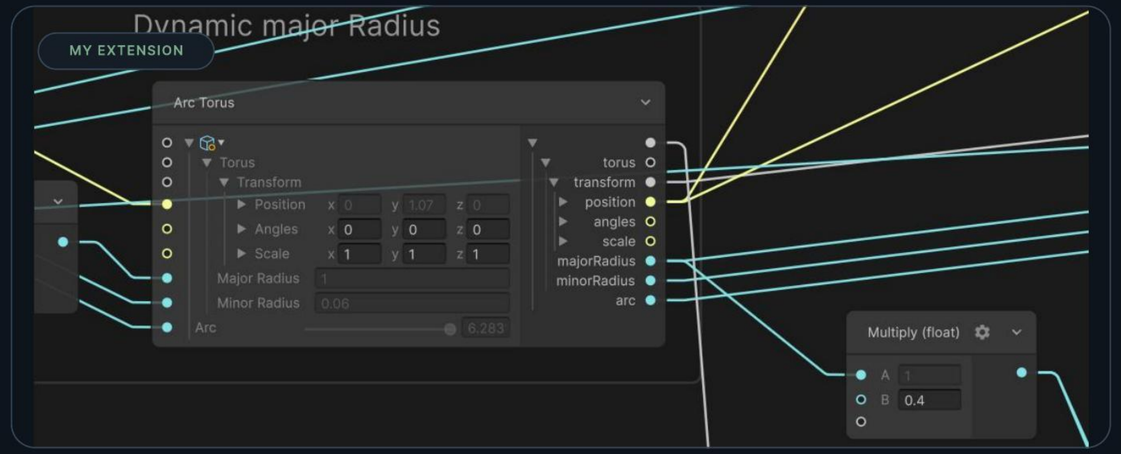
Scope, Attribution & Extension

基于 Unity 官方 Portal Sample，聚焦我实际完成的分析、调试、调参与 Formation 扩展。

ORIGINAL EXTENSION / REAL GRAPH EVIDENCE

Formation grows with the torus.

加入了 Formation Progress 驱动的 Major Radius 增长，让 Portal 的形成过程从“出现”变成“生长”。



FORMATION DATA FLOW

Formation Progress → **Major Radius** → **Torus Growth**

目标不是新增复杂系统，而是让 Formation 的 Shape Evolution 更明确。

RESPONSIBILITY BOUNDARY

Reference foundation: Unity Technologies / VisualEffectGraph-Samples / Portal。

Unity 官方 Sample 提供基础系统；以下只列本人实际完成、并有 Evidence 支撑的工作。

01 **System analysis & adaptation**

梳理 VFX Graph / Shader Graph 的职责与数据关系，并按真实 Evidence 重组 Case Study。

02 **Distortion debugging**

验证 Mask → Offset → Screen Position → Scene Color，并定位 UV0 / Screen UV 的语义错误。

03 **Motion tuning**

围绕 Pulse 与 Turbulence 调整运动节奏，并用连续 Runtime 帧验证结果。

04 **Torus formation extension**

新增 Major Radius 随 Formation Progress 增长，强化 Portal 从小到大的形成过程。

05 **Camera & presentation**

完成 Capture、Debug Visualization 与 Technical Editorial 呈现。

REFERENCE FOUNDATION

Unity Technologies · VisualEffectGraph-Samples / Portal ↗

TECH STACK Unity · Visual Effect Graph · Shader Graph · URP · C#

UNITY 6 URP VFX GRAPH C#

Ground Slash VFX

Unity 6 URP / VFX Graph 地面斩击技能特效，围绕视觉层次、地表反馈与可复用 Runtime 结构完成迭代。



FINAL RESULT / HERO

PROJECT FOCUS

以清晰的斩击主体为视觉锚点，通过连续地表热痕与短促碎石反馈强化运动方向和落地冲击，并将原型整理为可独立触发、自动贴地移动的 Runtime VFX。

MY WORK

VFX Graph 分层与视觉迭代 · URP Decal 调试 · Runtime Ground Follow · 可复用触发结构 · Showcase Presentation

Layered Effect Breakdown

将同一运行时技能拆为 Main Slash、Ground Trail 与 Debris 三个职责层，分别控制形体、轨迹与冲击，再通过主次关系完成合成。



三个 Layer 独立控制后，在 Runtime 中共享同一前进方向与地表路径；最高亮度留给 Slash，Trail 延续运动，Debris 只承担次级冲击。

01



PRIMARY SHAPE

MAIN SLASH

暖白 Core 与橙色 Glow 建立最高亮度层，优先保证斩击轮廓与前进方向可读。

02



SURFACE TRACE

GROUND TRAIL

连续热痕把瞬时斩击延伸为可追踪的地表路径，承担接地感与短时视觉残留。

03



SECONDARY IMPACT

DEBRIS

碎石补充破地反馈与尺度感，同时控制数量和亮度，避免与主 Slash 竞争视觉焦点。

Runtime System & VFX Structure

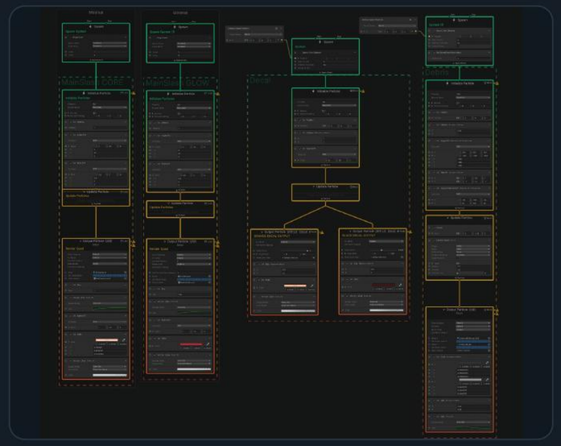
这一页关心的不是制作步骤，而是系统结构：触发、贴地运动与视觉合成被刻意拆开管理，因此这套效果既能独立迭代，也能被后续角色技能直接复用。

VFX GRAPH OVERVIEW

Layered systems, shared motion

Main Slash 建立第一视觉中心，Ground Trail 延续运动路径，Debris 补充落地冲击。三层保持独立控制，同时共享同一运动方向与地表关系。

- **Visual hierarchy**
  Slash 保持最高识别度，Trail 延长动作，Debris 只补充次级反馈。
- **Independent tuning**
  每一层都可以单独调整形体、亮度和密度，而不破坏其他层级。
- **Shared motion**
  三层最终服从同一前进方向，因此视觉拆分不会破坏动作连续性。

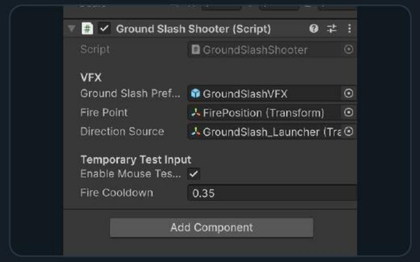


RUNTIME INTEGRATION

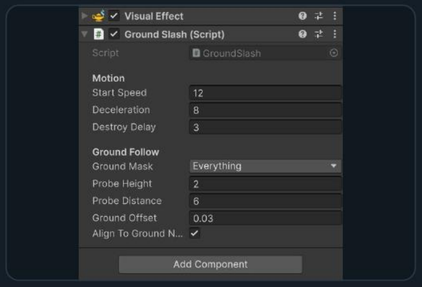
CAMERA-INDEPENDENT

Reusable runtime setup

01 Launcher — 02 Spawn / Launch — 03 Ground Follow



FirePoint 与 Direction Source 定义技能的出生条件，视觉效果本身不绑定 Camera 或控制器。



Spawned effect 独立维护移动与贴地状态，因此同一套 VFX 可以被不同角色或触发方式复用。

01 · MAIN SLASH

SHAPE / COMPOSITE



Primary Shape & Energy

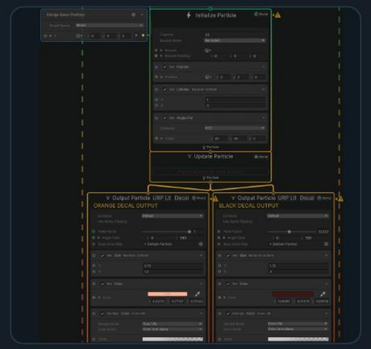
Core 保持清晰轮廓，Glow 提供能量外扩。两者共享同一形体基础，让亮度增强不会牺牲主体识别。

VISUAL ROLE

负责技能的第一视觉中心与方向识别。

02 · GROUND TRAIL

WORLD POSITION / DECAL



Persistent Surface Motion

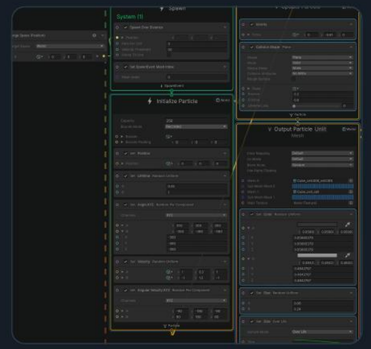
Ground Trail 将瞬时斩击转化为持续可见的地表路径，让运动方向在主体离开后仍然成立。

VISUAL ROLE

延续运动节奏，并连接 Slash 与地面反馈。

03 · DEBRIS

MOTION / COLLISION



Debris closes the hit

Debris 不抢主视觉，它只负责补上地面被击中的触感，让整个挥击有一个可信的收尾。

VISUAL ROLE

提供短促的物理反馈与尺度感。

Debugging Through Evidence

两个问题最终都不是表面现象本身：Decal 的可读性来自层级权重，Debris 的生成依赖正确的空间数据。这里保留的是定位过程中的关键证据与判断结果。

CASE 01 · DECAL READABILITY

Readability was a layering issue

BEFORE



AFTER



ROOT CAUSE EVIDENCE



DIAGNOSIS

Hot Core 的颜色本身没有失效。问题来自 Burn Layer 的 Fade Factor 权重过高，使上层暖色信息被持续压低。

OUTCOME

调整层级权重后，烧灼残留仍然保留，但 Hot Core 的亮度和识别度重新建立。

CASE 02 · SPAWN OVER DISTANCE

World motion was not reaching the spawn logic

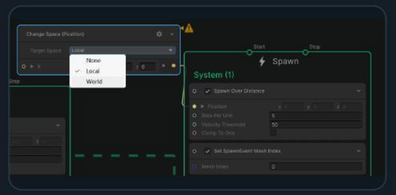
BEFORE



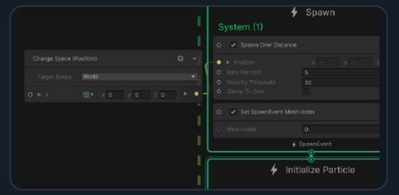
AFTER



WRONG SPACE



CORRECT SPACE



DIAGNOSIS

Ground Slash 在场景中确实持续移动，但 Spawn Over Distance 接收到的仍是 Local Position，因此输入值没有反映实际的世界空间位移。

OUTCOME

Position 转换到 World Space 后，移动距离重新成为有效输入，Debris 能稳定沿技能路径生成。

SCOPE & MY WORK / 项目范围与本人工作

以教程完整复现为起点；以下为本人后续完成的适配、调试、运行时重构与展示工作。

- Unity 6 / URP 适配与 VFX Graph 调试
- Camera-independent Launcher 与 Ground Follow 运行时结构
- Slash / Trail / Debris 的视觉层级与表现调整
- 自定义 Mesh 制作、场景整合与最终 Showcase

ATTRIBUTION / 资源说明

Effect Reference

Unity VFX Graph - Ground Slash Tutorial——Gabriel Aguiar Prod.↗

完整效果结构以该教程为学习与复现基础。

Environment

Inferno World Free — ITHappy Studios↗

用于最终 Showcase 场景与环境表现。

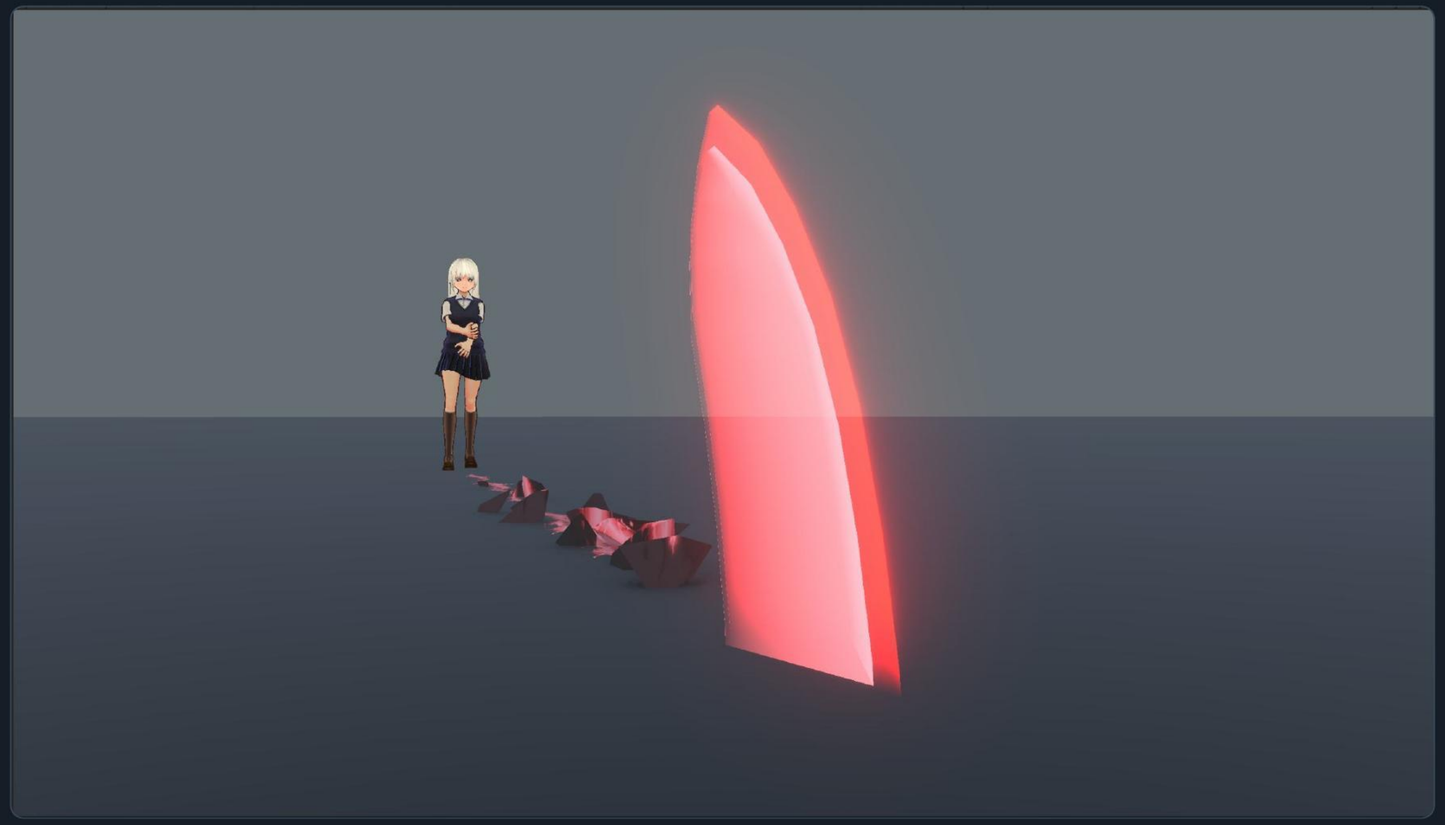
Custom Assets

项目中的自定义 Mesh 由本人使用 Blender 制作。

Character Skill Integration

通过 Animation Event 将角色攻击动作接入可复用的 Ground Slash Runtime VFX

FINAL RESULT / CHARACTER INTEGRATION



Animation Event 在攻击动作的关键帧触发技能；Launcher 只接收生成位置与方向，不依赖 Camera 或特定角色控制器。生成后的 Ground Slash 实例独立维护前进运动与 Ground Follow。

RUNTIME PIPELINE

Animation → Event → Controller → Launcher → VFX

Animation	Event	Controller	Launcher
Attack clip	FireGroundSlash	技能调用桥接	Spawn + direction

EVIDENCE 01 · ANIMATION EVENT



EVIDENCE 02 · CONTROLLER BRIDGE

```
public void FireGroundSlash()
{
    if (groundSlashShooter == null)
    {
        Debug.LogError(
            "CharacterSkillVFXController: GroundSlashShooter is not assigned.",
            this
        );
        return;
    }

    groundSlashShooter.Fire();
}
```

TECHNICAL ART CASE STUDY · Summary

Ground Slash Technical Summary

从 VFX Graph 原型、运行时地面跟随，到 Animation Event 角色接入，完成一个可复用的 Stylized Skill Module。

FROM SAMPLE TO CHARACTER SKILL



PROJECT EVOLUTION

1. **Tutorial Reconstruction**
   完整实现 Ground Slash 特效结构，建立 VFX Graph 基线。
2. **Runtime Refactor**
   将触发、移动与视觉拆开，建立 camera-independent Launcher 与 Ground Follow。
3. **Character Variant**
   重新调整 Slash / Ground Trail / Debris 的视觉层级与角色配色。
4. **Animation Integration**
   通过 Animation Event 将攻击动作接入 Runtime VFX。

WHAT I BUILT

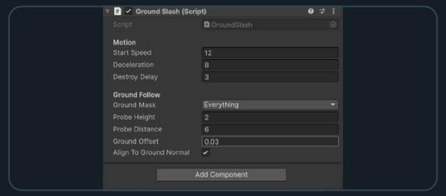
- **Layered Stylized VFX**
  Main Slash、Ground Trail 与 Debris 分层控制，分别承担方向识别、运动延续与冲击反馈。
- **Reusable Runtime Structure**
  Launcher 仅提供生成位置与方向；生成后的实例独立处理前进运动和 Ground Follow。

RUNTIME ARCHITECTURE

Animation → Event → Controller → Launcher → VFX

Animation	Event	Controller	Launcher	VFX
Attack Clip	FireGroundSlash	Bridge Call	Spawn + Direction	Move + Follow

TECHNICAL PROOF




```
public void FireGroundSlash()
{
    if (groundSlashShooter == null)
    {
        Debug.LogError(
            "CharacterSkillVFXController: GroundSlashShooter is not assigned.",
            this
        );
        return;
    }

    groundSlashShooter.Fire();
}
```

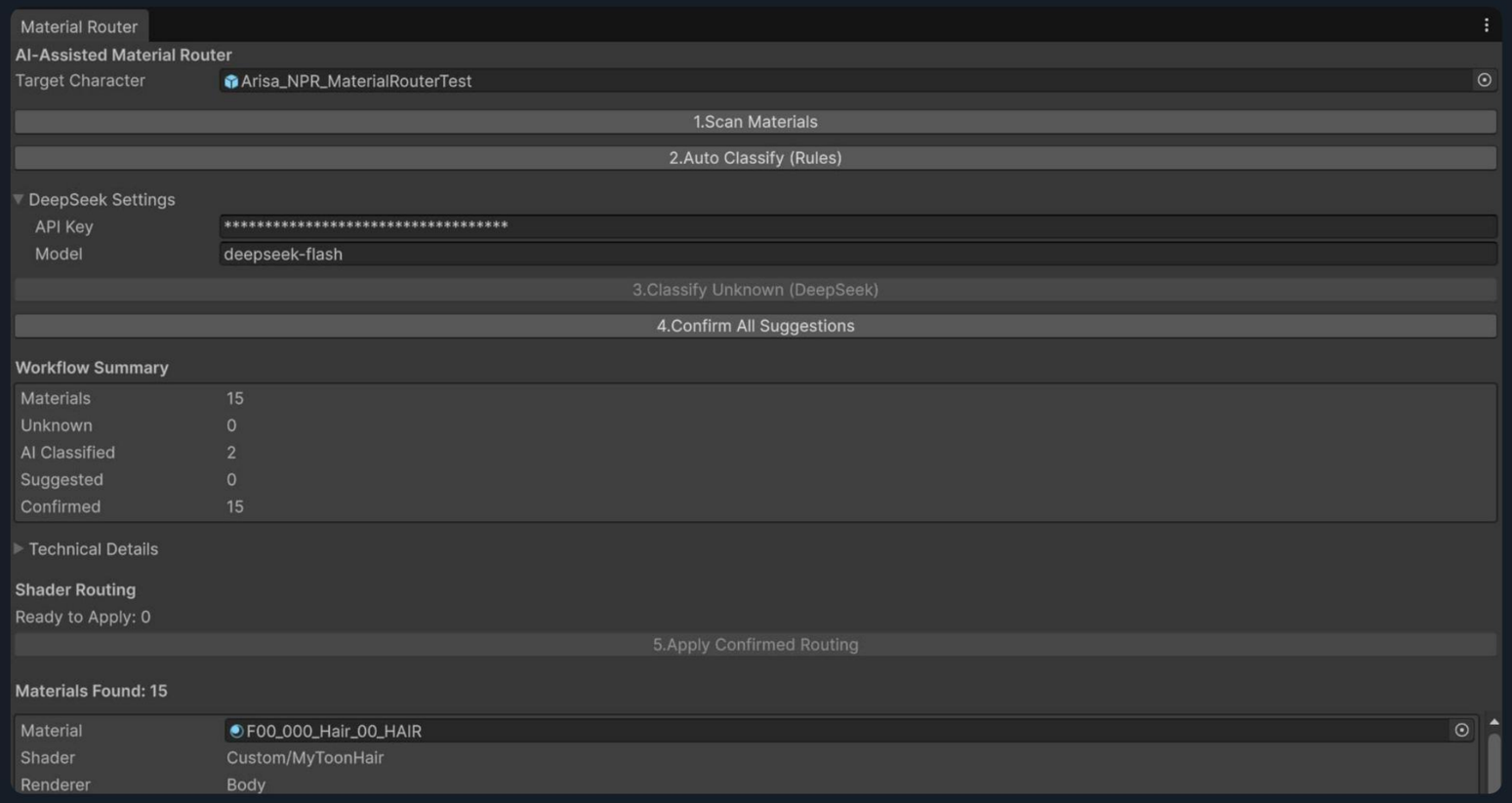
WHAT IT DEMONSTRATES

Stylized VFX · VFX Graph · Runtime C# · Animation Integration · Ground Follow · Shader / Decal Iteration · Debugging by Evidence · Portfolio Documentation

AI-Assisted Material Router

规则优先处理明确材质，仅将模糊项交给 LLM，并在人工确认后执行可撤销的 Shader Routing。

FINAL TOOL / MATERIAL ROUTING WORKFLOW



15 MATERIALS　2 AI FALLBACK　15 CONFIRMED　UNDO-SAFE APPLY

PROJECT FOCUS

Hybrid classification without black-box automation

将确定性规则作为第一层，AI 仅处理 Unknown；分类结果在进入资产修改前必须经过人工确认。

MY WORK

- Material Context 收集：Renderer / Slot / Texture
- Rule-based 分类与 Unknown 检测
- DeepSeek structured fallback + AI Reason
- Human Review / Routing Preview / Undo-safe Apply

STACK

Unity 6　C#　Editor Tooling　DeepSeek API　Human-in-the-loop　Shader Routing　Undo

Hybrid Classification Workflow

Rules first, AI only for ambiguity. 规则先解决确定项，LLM 只处理 Unknown，并在人工确认后进入 Routing。

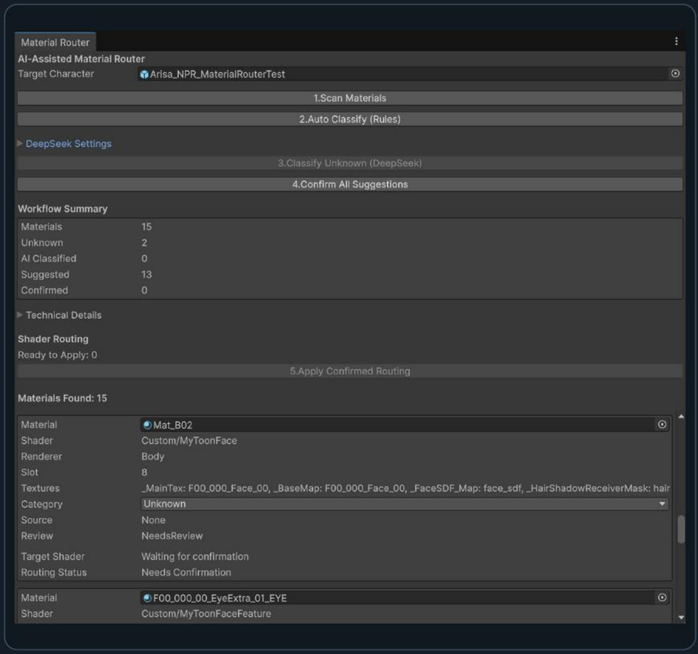
WORKFLOW

01 SCAN — 02 RULES — 03 UNKNOWN — 04 DEEPSEEK — 05 HUMAN REVIEW

EVIDENCE 01 · RULE PASS

13 resolved · 2 Unknown

15 个材质中，规则层先完成 13 个高置信度建议，仅保留 2 个模糊项。

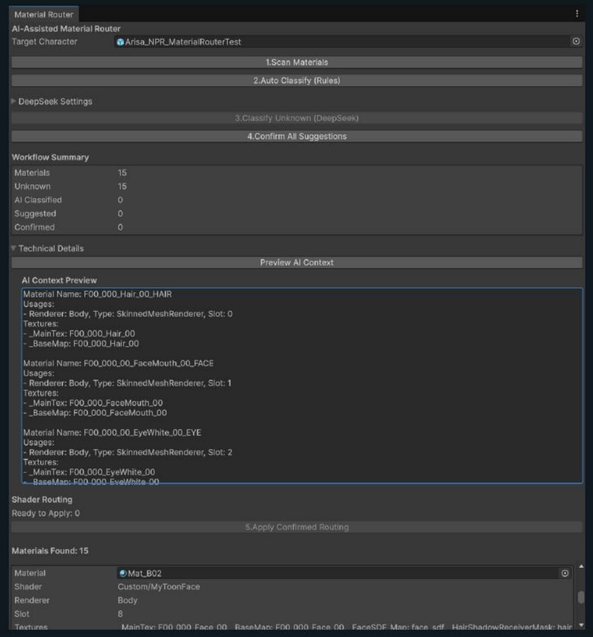


RULE-BASED

EVIDENCE 02 · AI INPUT

Context sent to LLM

Material / Renderer / Slot / Texture 构成可检查的 AI Context，不把 Current Shader 当答案。

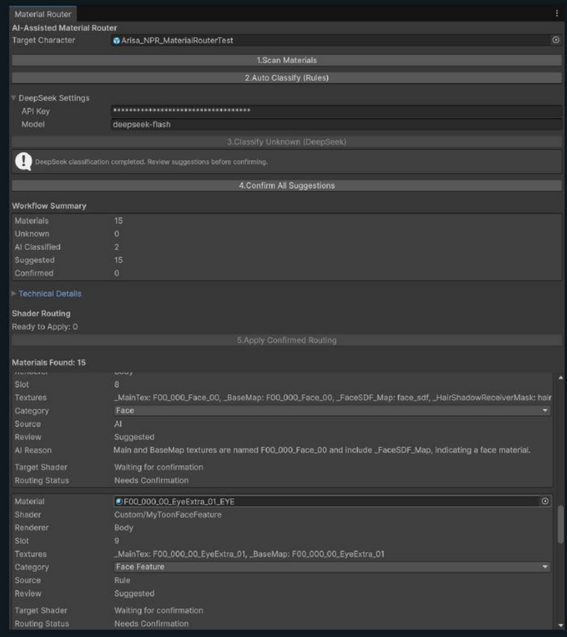


CONTEXT-AWARE

EVIDENCE 03 · AI OUTPUT

Structured suggestion

返回 Category + Source + Review + Reason；结果仍是 Suggested，等待人工确认。



HUMAN-REVIEWED

TECHNICAL ART CASE STUDY · SAFE SHADER ROUTING

Preview Before Apply

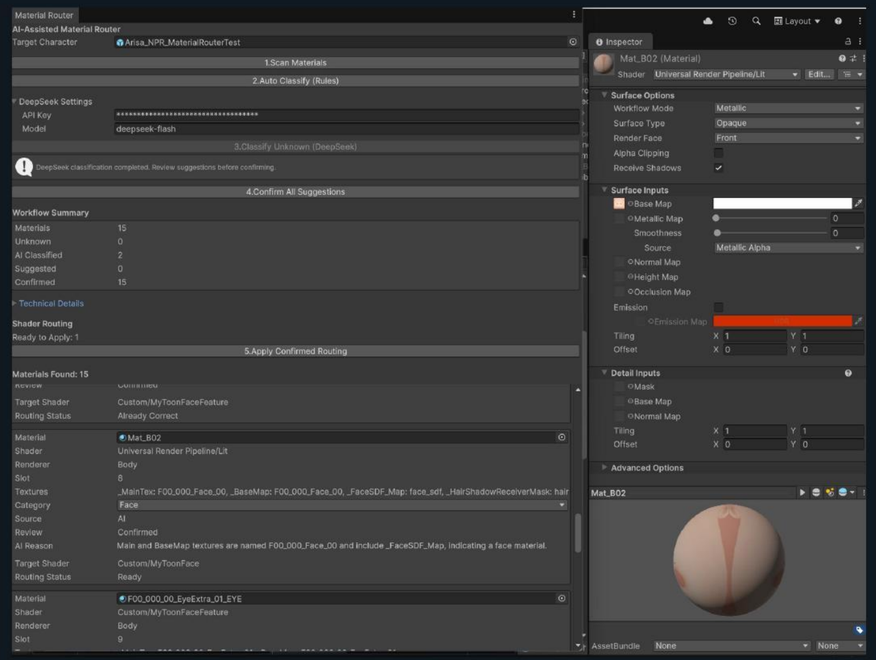
分类结果不会直接修改资产：先确认 Category，再预览 Current / Target Shader，最后执行可撤销的 Apply。

01 CONFIRMED CATEGORY → 02 ROUTING PREVIEW → 03 HUMAN APPLY → 04 UNDO-SAFE RESULT

BEFORE APPLY · ROUTING PREVIEW

Current Shader ≠ Target Shader

Mat_B02 已由 AI 分类为 Face，并完成人工确认；工具只标记 Ready，不在预览阶段修改资产。



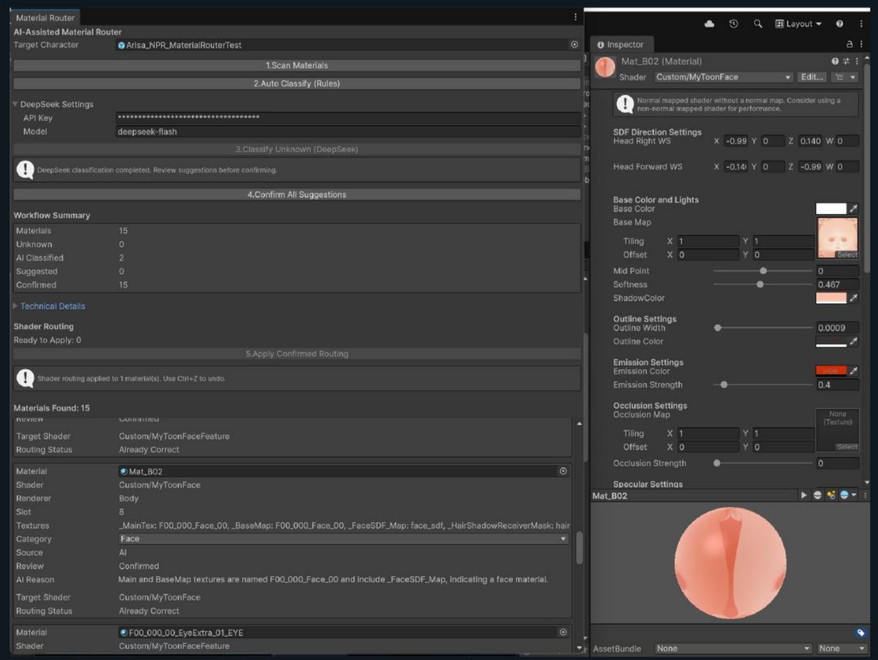
READY TO APPLY · 1

STATUS · READY

AFTER APPLY · VERIFIED RESULT

Target Shader Applied

执行 Apply 后，材质切换到 Custom/MyToonFace；状态回到 Already Correct，并保留 Ctrl+Z 撤销能力。



READY TO APPLY · 0

STATUS · ALREADY CORRECT

CONFIRMED ONLY · PREVIEW BEFORE MODIFICATION · UNDO.RECORDOBJECT() · SAFE APPLY

HUMAN CONFIRMATION · ROUTING PREVIEW · UNDO-SAFE APPLY

TECHNICAL ART CASE STUDY · ARCHITECTURE & SCOPE

Architecture, Scope & V1 Boundaries

把确定性分类、LLM fallback、人工审核与资产修改拆成独立层；AI 是模糊信息处理器，而不是整个工具的控制中心。

SYSTEM ARCHITECTURE

Rules first → AI for ambiguity → Human confirmation → Safe asset change



V1 · COMPLETED LOOP

What the tool already proves

- **Context-aware scanning**
  遍历角色 Renderer 与 sharedMaterials，并记录 Slot / Texture Context。
- **Hybrid classification**
  规则优先；只有 Unknown 才调用 DeepSeek，降低成本与随机性。
- **Human-reviewed routing**
  Source / Review 状态区分 Rule、AI、Manual，并要求确认后才进入 Routing。
- **Undo-safe apply**
  Current / Target Shader 对比、Ready 状态、Undo.RecordObject() 安全执行。

V1 · SCOPE BOUNDARIES

Deliberately not solved yet

- **Fixed categories**
  当前 Category 集合固定，Rule 仍针对现有角色命名习惯。
- **Hard-coded routing table**
  Category → Shader 映射仍写在 EditorWindow 逻辑中。
- **Shader switch, not conversion**
  V1 只切换 Shader，不迁移完整 Properties / Keywords / Presets。
- **Single-character workflow**
  一次处理一个 Target Character；结果尚未持久化成独立数据库。

NEXT · EXTENSION PATH

Natural V2 extensions

- **Configurable routing**
  将 Category / Shader Mapping 外置成配置或 Preset。
- **Property migration**
  补充旧 Shader → NPR Shader 的参数、贴图与 Keyword 映射。
- **Provider abstraction**
  把 LLM Provider 从 EditorWindow 解耦，支持 DeepSeek / Gemini / OpenAI 等。
- **Batch + reporting**
  多角色批处理、结果持久化、错误报告与可重复执行。

UNITY 6 · C# · EDITOR TOOL · DEEPSEEK · RULES FIRST · HUMAN REVIEW · SHADER ROUTING · UNDO-SAFE APPLY

DETERMINISTIC BASELINE · LLM FALLBACK · HUMAN-IN-THE-LOOP · SAFE ASSET MODIFICATION